Aegis - How It Works (Plain Terms)

Aegis is a safety gate between an AI agent and your kdb+ systems. It never runs the AI's code - it reads the intent, rebuilds the query from a fixed menu of safe parts, and runs only its own rebuild. Anything it can't rebuild is refused.

The core idea

The AI can *write* q, but that text never reaches the database. Aegis parses it and re-generates a fresh, bounded query from approved building blocks. It is an **allowlist, not a blocklist** - enumerate what's *good*, refuse everything else. You can't out-clever a closed menu.

The flow of one query

1. You ask the chatbot a question. **Claude (the LLM)** writes some q and asks to run a **tool**.
2. Aegis intercepts before anything executes (`server.py:_run_tool` → `govern`).
3. **q → Python:** `lift()` parses the q *string* into a structured dictionary - `{table, columns, aggs, where}`. Any off-menu token (`delete`, `.`, `[`, `;`) → it can't be placed on the ticket → **refused**.
4. **Python → q:** `compile()` checks every field against the menu and **writes a brand-new q string**, stamping on the safety bounds (row cap, date-first). Values are re-serialised *by type*, so text can never become code.
5. **Run:** only the rebuilt string is sent to kdb+ via **PyKX**. **kdb+ executes it - not Python.** PyKX returns the result as a table.

> PyKX is only the *pipe* to kdb+ at step 5. The gate itself (lift + compile) is pure Python string work - no PyKX, no execution.

Two layers you can change

- **The menu - JSON, no code.** `aegis_policy.fsp.json`: allowed tables, columns, aggregations, row caps, and which **tools** the agent may call. Add/remove = edit the JSON + restart (in production, re-sign it). Widening it only ever expands *bounded, read-only* access - it can never create a dangerous operation.
- **The grammar - code + tests.** The *shapes* of query understood (`select/by/where`, null-counts, recent-window filters) live in `lift()` + `compile()`. A new shape = a small parser rule + a small emitter rule + **adversarial tests**, keeping the acceptance suite green.

What "tools" are

The named functions the AI is allowed to request - `query_fsp_data`, `check_data_coverage`, `restart_process`, etc. The AI can't touch kdb+, files, or the shell directly; it can only *ask* for a tool, and every request still passes through Aegis. `grants.tools` lists which tools even exist - remove one and the AI simply cannot use it.

How "good code" is enforced (the guarantees)

Every query that reaches kdb+ is, **by construction:**

- **read-only** - `delete`/`update`/`system` aren't in the grammar, so they can't be written;
- **on approved data** - table/column checked against the allowlist;
- **bounded** - caps are auto-added, so a query can't run away and OOM the box;
- **injection-proof** - values re-typed, so a string can't smuggle code.

Fail-closed: any error or ambiguity → block. And because the AI's raw text is *never executed* - only Aegis's rebuild - even a parser bug can at worst *refuse*, never run something dangerous.

What it is NOT (honest limits)

- **Not a correctness oracle** - it guarantees *safe*, not *right*. The AI can still write a valid query and draw a wrong conclusion.
- **Not all of q** - it covers the safe analytic subset; the rest is refused and grown deliberately (the "couldn't-lift" list is the backlog).
- **Hardened vs embedded** - the production form is out-of-process + signed + read-only (can't be disabled). An in-process embed runs the same logic but without the OS-level can't-be-disabled property.

One line: *Read the intent → rebuild it from a safe menu (plus bounds) → run only the rebuild; refuse anything off-menu. The menu is JSON you edit; the query shapes are code you extend with tests.*